

CNN1D Model

Layer & Parameter Reference

Automatic Modulation Classification · RadioML 2016.10a / 2018.01a



This document is a layer-by-layer reference for the CNN1D model defined in **models/cnn1d.py**. Every layer, every parameter, and every design choice is explained in plain language so you can reason about what the network is doing at each stage — and why it is built that way. The original model code is reproduced verbatim at the end for reference.

Contents

1. `keras.Input` — Entry point
2. Conv1D Block 1 — Local feature detection (64 filters)
3. MaxPooling1D + BatchNormalization — Compression & stability
4. Conv1D Block 2 — Higher-level features (128 filters)
5. MaxPooling1D + BatchNormalization — Second compression
6. Flatten — Unrolling into a 1-D vector
7. Dense(256, relu) — Global feature combination
8. Dropout(0.5) — Regularisation
9. Dense(11, softmax) — Output / classifier
10. Compile settings — Optimizer, loss, metrics
11. Full model code

1. `keras.Input`

Entry point — defines what data the model accepts

```
inputs = keras.Input(shape=(128, 2), name='iq_input')
```

What it does. `keras.Input` creates the symbolic entry point of a Functional-API model. It does not hold trainable weights — it simply declares the shape and dtype of the data the model will receive.

The Functional API (as opposed to the Sequential API) is used here because it allows branches, skip connections, and multiple inputs or outputs — all of which are needed for more advanced architectures like ResNet. Building every model in the Functional style from the start keeps the codebase consistent.

Parameters

shape		Dimensions excluding the batch axis. 128 = number of I/Q time samples in one signal window. 2 = two channels: I (in-phase) and Q (quadrature). Every radio sample is a complex number; splitting into I and Q lets a real-valued network process it.
name		Optional string label. Keras uses this when printing <code>model.summary()</code> and when saving/loading the model. Naming layers is good practice for debugging.

Note: the batch dimension is never included in shape. Keras prepends it automatically. A shape of (128, 2) means each training example is 128 timesteps of 2 values — the network processes a batch of these at once.

2. Conv1D Block 1

conv1_1 and conv1_2 — local feature detection with 64 filters

(batch, 128, 2) → (batch, 128, 64) → (batch, 128, 64)

```
x = layers.Conv1D(
    filters=64,
    kernel_size=3,
    padding='same',
    activation='relu',
    kernel_regularizer=regularizers.l2(1e-4),
    name='conv1_1',
)(inputs)
x = layers.Conv1D( # identical settings
    filters=64, kernel_size=3, padding='same',
    activation='relu', kernel_regularizer=regularizers.l2(1e-4),
    name='conv1_2',
)(x)
```

What it does. A 1-D convolution slides a small learnable filter across the time axis and produces an activation map. You can think of each filter as a pattern detector: it fires strongly when it recognises a particular local shape in the signal (e.g. a sharp phase transition, a repeating oscillation, or a specific amplitude envelope) at some position in the 128-sample window.

Two Conv1D layers are stacked before the first pooling step. The first layer (conv1_1) detects primitive local patterns directly from the raw I/Q data. The second layer (conv1_2) sees the output of conv1_1 and therefore learns combinations of those primitives — more abstract features — before any spatial compression happens. Stacking two convolutions doubles the effective receptive field (each output cell now looks at 5 input timesteps: 3 from conv1_2 centred on 3 from conv1_1) without requiring a larger, more expensive kernel.

Parameters

filters

Number of independent pattern detectors (output channels). Each filter has its own learnable weights. The output tensor gains 64 channels: one activation map per filter. 64 is a standard starting size — rich enough to learn varied patterns but light enough to train quickly.

kernel_size		How many consecutive timesteps each filter covers. A kernel of 3 means: at every position t , the filter multiplies weights with samples $[t-1, t, t+1]$ and sums them. Small kernels (3, 5) capture fine local structure. Larger kernels (7+) capture longer dependencies but add parameters. For radio signals at 128 samples, size 3 works well because the relevant patterns (symbol edges, carrier phase) are short.
padding		Pads the input edges with zeros so the output length equals the input length (128 stays 128 after every Conv1D). The alternative, 'valid', shrinks the sequence by $(\text{kernel_size} - 1)$ at each layer, which makes spatial bookkeeping harder and discards information at the edges.
activation		Applies the Rectified Linear Unit — $f(x) = \max(0, x)$ — element-wise after the dot product. Without a non-linearity, stacking multiple Conv layers is mathematically equivalent to a single linear layer. ReLU is cheap, does not saturate for large positive values, and has been shown to train faster and generalise better than sigmoid or tanh for deep networks.
kernel_regularizer		Adds a penalty of $\lambda * \sum(w^2)$ to the training loss, where $\lambda = 0.0001$. This discourages very large weight values, which are associated with overfitting. The effect is mild at $1e-4$ — strong enough to regularise without strangling learning. <code>use_bias</code> defaults to <code>True</code> : each filter also has an additive bias term.

3. MaxPooling1D + BatchNormalization

Spatial compression and activation normalisation after Block 1

(batch, 128, 64) → **(batch, 64, 64)** → **(batch, 64, 64)**

```
x = layers.MaxPooling1D(pool_size=2, name='pool1')(x)
x = layers.BatchNormalization(name='bn1')(x)
```

MaxPooling1D. Splits the time axis into non-overlapping windows of `pool_size=2` and keeps only the maximum activation in each window. This halves the sequence length ($128 \rightarrow 64$) while retaining the strongest evidence of each pattern. Two benefits: (1) the subsequent layers see a shorter, cheaper-to-compute sequence; (2) slight translational invariance — a feature that shifts by one sample still falls in the same pool window and produces the same output.

BatchNormalization. After pooling, the activations are re-normalised so that, across the batch, each feature channel has roughly zero mean and unit variance. It has four learnable

parameters per channel: a scale (gamma) and shift (beta) that let the network un-normalise if needed, plus running mean and variance used at inference time. Benefits: prevents any one layer from saturating the next, allows higher learning rates, and provides mild regularisation. It is placed after activation here (post-activation BN), which is the most common convention.

4. Conv1D Block 2

conv2_1 and conv2_2 — higher-level features with 128 filters

(batch, 64, 64) → (batch, 64, 128) → (batch, 64, 128)

```
x = layers.Conv1D(
    filters=128, kernel_size=3, padding='same',
    activation='relu', kernel_regularizer=regularizers.l2(1e-4),
    name='conv2_1',
)(x)
x = layers.Conv1D(
    filters=128, kernel_size=3, padding='same',
    activation='relu', kernel_regularizer=regularizers.l2(1e-4),
    name='conv2_2',
)(x)
```

What changed from Block 1. The only difference is `filters=128`. Everything else — kernel size, padding, activation, regulariser — is identical. Doubling the filter count follows the standard CNN design principle: as the spatial dimension shrinks (via pooling), the channel depth grows to compensate. The network can afford more filters now because each feature map is only 64 timesteps long instead of 128. The 128 filters learn higher-level combinations of the 64 Block 1 features — they can encode properties like modulation order, symbol rate, and inter-symbol relationships that span multiple Block 1 activations.

After two stacked Conv1D layers in this block, the receptive field of a single output neuron covers 5 timesteps at Block 2 resolution, which corresponds to 10 original I/Q samples (because pooling doubled the stride). This is long enough to see full symbol transitions in most common modulation types at the standard RadioML sample rate.

5. MaxPooling1D + BatchNormalization (Block 2)

Second spatial compression — sequence length 64 → 32

(batch, 64, 128) → (batch, 32, 128) → (batch, 32, 128)

```
x = layers.MaxPooling1D(pool_size=2, name='pool2')(x)
x = layers.BatchNormalization(name='bn2')(x)
```

Identical in structure to Section 3. After this step the tensor is (batch, 32, 128): 32 timesteps, each described by 128 learned feature values. This is the most compressed representation before the classifier head. The two rounds of pooling mean that each of the 32 positions summarises a window of 4 original I/Q samples, and each of the 128 channels encodes a high-level pattern detected across those samples.

6. Flatten

Unrolling the 2-D feature map into a 1-D vector for the Dense layers

(batch, 32, 128)

→

(batch, 4096)

```
x = layers.Flatten(name='flatten')(x)
```

What it does. Flatten has no learnable parameters. It concatenates all time positions and all channels into a single vector: $32 \times 128 = 4096$ values. Dense layers require a 1-D input per example, so Flatten is the necessary bridge between the convolutional feature extractor and the classifier head.

Trade-off vs GlobalAveragePooling1D. Flatten preserves all spatial information at the cost of a large vector (4096). GlobalAveragePooling1D (used in ResNet) would instead average across the 32 positions, producing a 128-dimensional vector. GAP is more parameter-efficient and less prone to overfitting, but may lose positional information that matters for discrimination. CNN1D uses Flatten because it is simpler and the 32-timestep sequence is short enough that 4096 inputs to the Dense layer is not prohibitive.

7. Dense(256, relu)

Fully-connected layer — learns global combinations across all features

(batch, 4096)

→

(batch, 256)

```
x = layers.Dense(256, activation='relu', name='dense1')(x)
```

What it does. A Dense (fully-connected) layer multiplies its input by a weight matrix of shape (4096, 256), adds a bias vector of length 256, then applies ReLU. Every one of the 4096 input values is connected to every one of the 256 output neurons. This allows the layer to learn arbitrary global combinations across all features and all time positions simultaneously — something Conv layers cannot do because they only look at local windows.

256 neurons is a reasonable bottleneck: wide enough to represent complex decision boundaries between 11 modulation classes, but narrow enough to force the network to compress and summarise rather than memorise. The parameter count for this layer alone is $4096 \times 256 + 256 = 1,049,088$ — this is the single most parameter-heavy layer in the model.

8. Dropout(0.5)

Regularisation — randomly disables neurons during training

(batch, 256)

→

(batch, 256) [same shape,
some zeros]

```
x = layers.Dropout(dropout_rate, name='dropout')(x) # dropout_rate=0.5
```

What it does. During each training step, Dropout randomly sets 50% of the 256 activations to zero and scales the remaining ones up by $1 / (1 - 0.5) = 2$ to keep the expected sum unchanged. A different random mask is used for every example in every batch.

Why it works. By randomly disabling neurons, Dropout forces every neuron to be useful on its own rather than relying on a fixed set of co-activating neighbours. The result is an implicit ensemble of 2^{256} different sub-networks that share weights — at test time the full network with halved weights approximates their average.

Important: Dropout is active only during training. It is automatically disabled when you call `model.evaluate()`, `model.predict()`, or pass `training=False`. You do not need to toggle it manually.

9. Dense(11, softmax)

Output layer — converts features to class probabilities

(batch, 256)

→

(batch, 11)

```
outputs = layers.Dense(num_classes, activation='softmax', name='output')(x)
# num_classes = 11 for RadioML 2016.10a
```

11 neurons correspond to the 11 modulation classes in RadioML 2016.10a (e.g. BPSK, QPSK, 8PSK, 16QAM, AM-DSB, ...). For 2018.01a, set `num_classes=24` and retrain.

Softmax takes the raw score (logit) for each class and converts the vector into a probability distribution that sums to 1. Formally, for class i : $p(i) = \exp(z_i) / \sum_j \exp(z_j)$. The predicted class is the index with the highest probability: `np.argmax(model.predict(x), axis=1)`.

10. Compile settings

Optimizer, loss function, and metrics used in train.py

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)
```

Parameters

Adam(lr=1e-3)		Adaptive Moment Estimation. Maintains a per-parameter learning rate that adapts based on first and second moment estimates of the gradient. Default lr=0.001 works well as a starting point. Reduce via ReduceLROnPlateau callback if training stalls.
categorical_crossentropy		The standard loss for multi-class classification when labels are one-hot encoded (which <code>keras.utils.to_categorical</code> produces). It measures the KL divergence between the true distribution (one-hot) and the predicted softmax distribution. Use 'sparse_categorical_crossentropy' if your labels are integer indices instead.
'accuracy'		Fraction of examples where <code>argmax(prediction) == argmax(true_label)</code> . This is the primary metric you will compare across CNN1D, ResNet, and CNN-LSTM. Consider also adding per-SNR accuracy via a custom callback for a complete picture.

11. Full model code

models/cnn1d.py — reproduced verbatim

```

def build_cnn1d(input_shape=(128, 2), num_classes=11, dropout_rate=0.5):
    inputs = keras.Input(shape=input_shape, name="iq_input")

    # Block 1
    x = layers.Conv1D(filters=64, kernel_size=3, padding="same",
        activation="relu", kernel_regularizer=regularizers.l2(1e-4),
        name="conv1_1")(inputs)
    x = layers.Conv1D(filters=64, kernel_size=3, padding="same",
        activation="relu", kernel_regularizer=regularizers.l2(1e-4),
        name="conv1_2")(x)
    x = layers.MaxPooling1D(pool_size=2, name="pool1")(x)
    x = layers.BatchNormalization(name="bn1")(x)

    # Block 2
    x = layers.Conv1D(filters=128, kernel_size=3, padding="same",
        activation="relu", kernel_regularizer=regularizers.l2(1e-4),
        name="conv2_1")(x)
    x = layers.Conv1D(filters=128, kernel_size=3, padding="same",
        activation="relu", kernel_regularizer=regularizers.l2(1e-4),
        name="conv2_2")(x)
    x = layers.MaxPooling1D(pool_size=2, name="pool2")(x)
    x = layers.BatchNormalization(name="bn2")(x)

    # Classifier head
    x = layers.Flatten(name="flatten")(x)
    x = layers.Dense(256, activation="relu", name="dense1")(x)
    x = layers.Dropout(dropout_rate, name="dropout")(x)
    outputs = layers.Dense(num_classes, activation="softmax",
        name="output")(x)

    model = keras.Model(inputs=inputs, outputs=outputs,
        name="CNN1D_AMC")

    return model

```